FINCOMERCIAL, S.A.

Analítica inteligente para anticipar el riesgo hipotecario

MANUAL DE DESPLIEGUE · FASE II

FinPredict

Manual técnico de despliegue y operación

PROPÓSITO DEL DOCUMENTO

Arquitectura, preparación, Docker Compose, verificación, operación, evidencias y propuesta de despliegue en AWS.

CAMPO	INFORMACIÓN
Organización	FinComercial, S.A.
Responsable	Lizbeth Bruno Lorenzana
Puesto	AI Lead
Asignatura	Gestión de Proyectos de Inteligencia Artificial
Solución	FinPredict – Fase II
Ámbito	Prototipo académico · Datos sintéticos
Versión documental	1.1 | Noviembre de 2026
Estado	Despliegue local validado; nube propuesta

Documento preparado para consulta, evaluación académica y trazabilidad técnica en el repositorio del proyecto.

FinComercial, S.A. Analítica inteligente para anticipar el riesgo hipotecario

Campo	Información
Proyecto	FinPredict – Fase II
Tipo de solución	Sistema de inferencia de inteligencia artificial
Modelo	Regresión logística
Arquitectura	Streamlit + FastAPI + Docker Compose
Modalidad validada	Despliegue local contenerizado
Modalidad propuesta	Despliegue en infraestructura cloud
Versión documental	1.0
Fecha	Noviembre de 2026
Clasificación	Prototipo académico con datos sintéticos

1. Propósito y alcance

1.1. Propósito

Establecer un procedimiento técnico reproducible para construir, ejecutar, verificar y administrar FinPredict mediante contenedores Docker. El manual está dirigido a personas con conocimientos básicos de desarrollo de software, administración de contenedores o integración de aplicaciones. Su objetivo es permitir que otro usuario pueda ejecutar la solución sin depender del entorno virtual de Python utilizado durante el desarrollo.

1.2. Alcance funcional

FinPredict permite:

- Capturar 15 variables predictoras de un crédito sintético.
- Enviar solicitudes HTTP desde Streamlit a FastAPI.
- Validar los datos recibidos mediante Pydantic y reglas financieras preliminares.
- Ejecutar inferencias con un modelo de regresión logística.
- Presentar la probabilidad de incumplimiento y su clasificación.
- Construir y ejecutar dos servicios mediante Docker Compose.

1.3. Alcance del despliegue

Despliegue validado: La solución fue construida y ejecutada en Docker Desktop, utilizando dos contenedores conectados mediante una red interna administrada por Docker Compose. Se comprobó la disponibilidad de los servicios, la comunicación frontend/backend y la consistencia de inferencia del registro sintético de referencia.
Despliegue no validado: No se ha realizado un despliegue público en infraestructura cloud ni se han implementado controles empresariales de alta disponibilidad, autenticación, autorización o escalamiento horizontal. El procedimiento cloud presentado posteriormente será una propuesta técnica de implementación y no una declaración de despliegue realizado.

1.4. Uso responsable

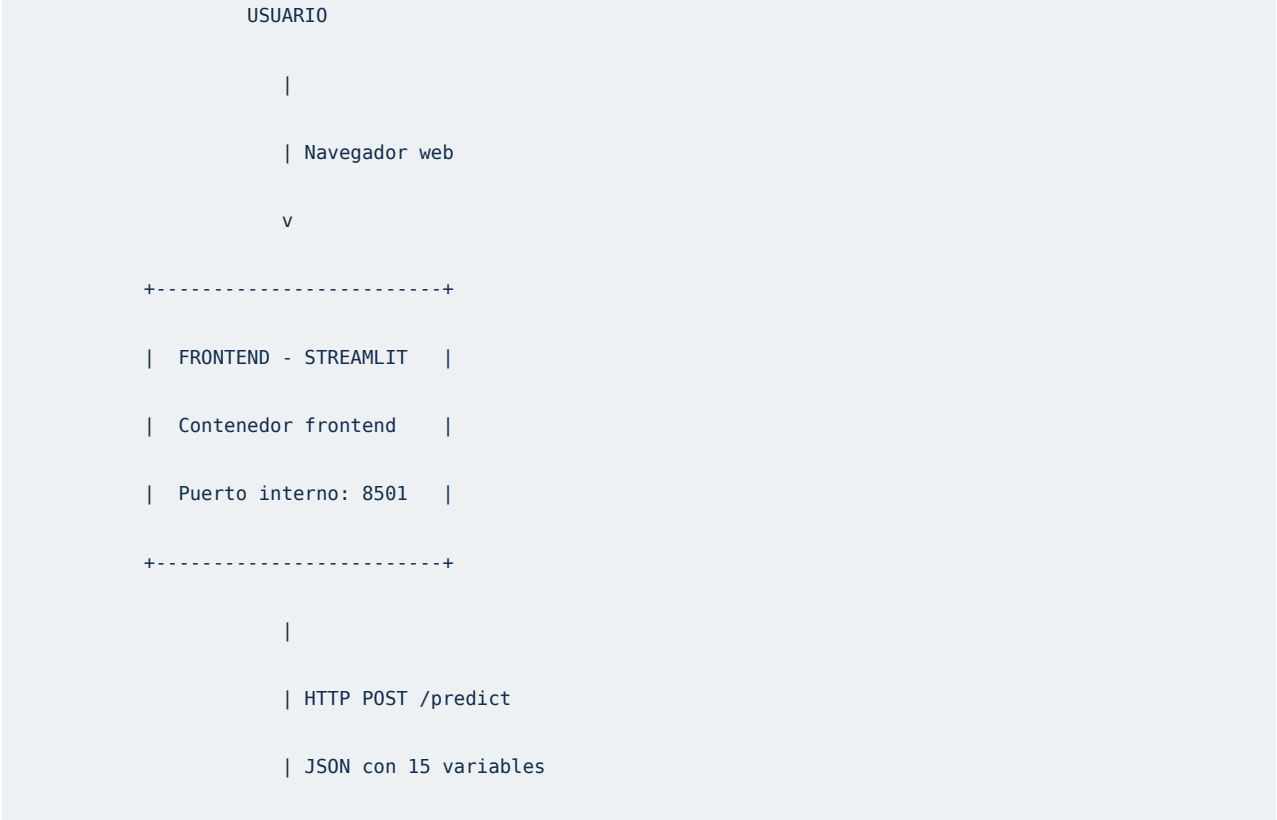
FinPredict utiliza exclusivamente información sintética para fines académicos y de validación técnica. Sus resultados no constituyen decisiones crediticias automatizadas ni deben utilizarse para aprobar, rechazar o modificar productos financieros reales. Antes de considerar una implementación productiva sería necesario evaluar la calidad de datos reales, el desempeño predictivo, la seguridad, el gobierno del modelo, el cumplimiento normativo y los riesgos operativos.

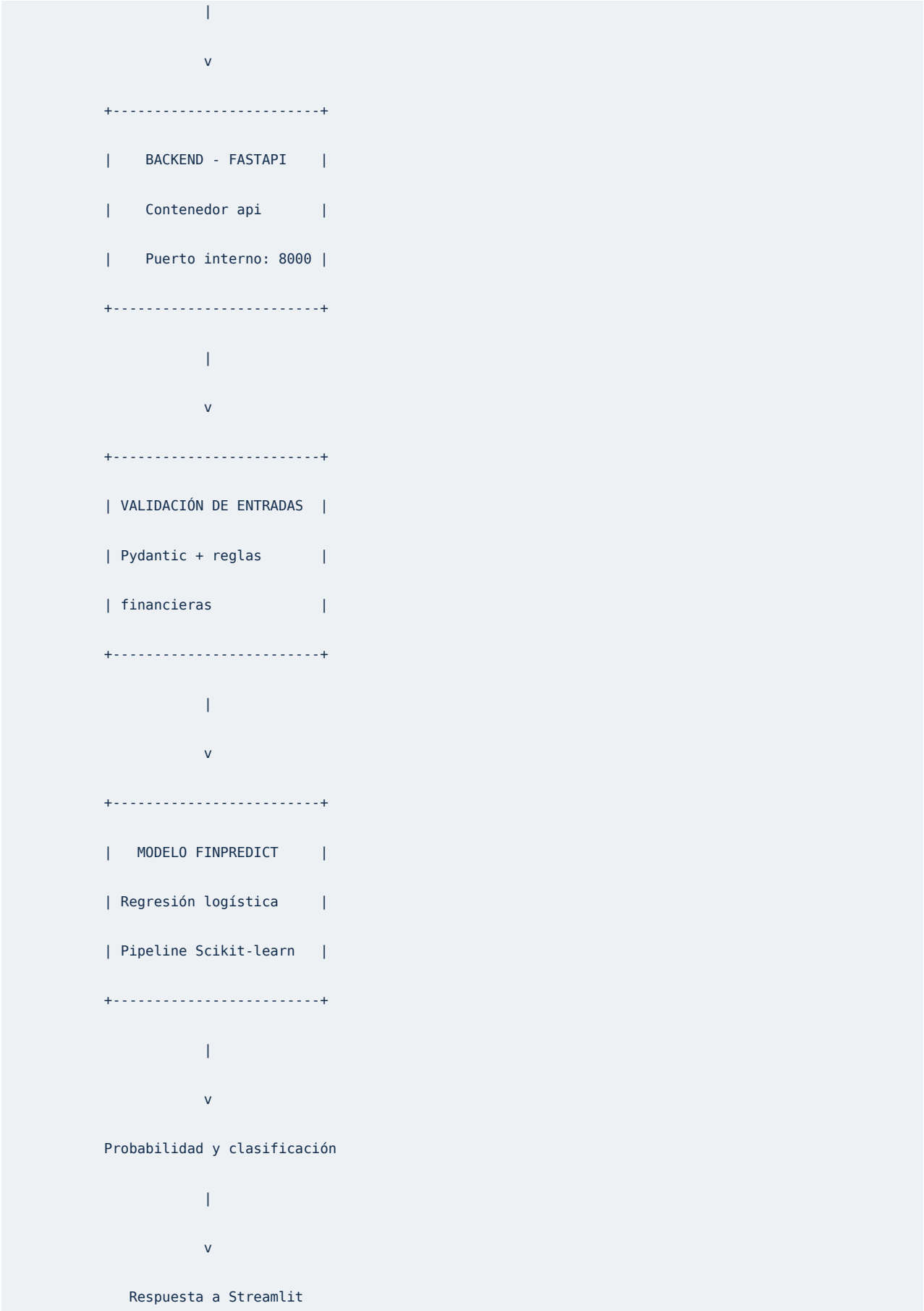
2. Arquitectura de la solución

2.1. Descripción general

FinPredict utiliza una arquitectura de dos servicios independientes, ejecutados mediante Docker Compose. El frontend proporciona una interfaz web para capturar información financiera y visualizar predicciones. El backend recibe las solicitudes, valida los datos, ejecuta el modelo y devuelve una respuesta estructurada. Ambos servicios se comunican mediante una red interna creada automáticamente por Docker Compose.

2.2. Flujo de comunicación




BACKEND - FASTAPI
Contenedor api
Puerto interno: 8000
VALIDACIÓN DE ENTRADAS
Pydantic + reglas
financieras
MODELO FINPREDICT
Regresión logística
Pipeline Scikit-learn
Probabilidad y clasificación
Respuesta a Streamlit

2.3. Componentes tecnológicos

Componente	Tecnología	Responsabilidad
Frontend	Streamlit	Captura y visualización
Backend	FastAPI	Servicio REST de inferencia
Validación	Pydantic	Tipos y campos obligatorios
Reglas financieras	Python	Restricciones preliminares
Modelo	Scikit-learn	Inferencia predictiva
Artefacto	Joblib	Persistencia del modelo
Contenedores	Docker	Entornos de ejecución
Coordinación local	Docker Compose	Construcción, red e inicio

2.4. Configuración de red

Los servicios utilizan los siguientes puertos:

Servicio	Puerto interno	Puerto en Windows
Frontend Streamlit	8501	8502
Backend FastAPI	8000	8002

Dirección de acceso al frontend: `http://127.0.0.1:8502` **Documentación interactiva de la API:** `http://127.0.0.1:8002/docs` **Endpoint de disponibilidad:** `http://127.0.0.1:8002/health` **Comunicación interna entre contenedores:** `http://api:8000/predict` El nombre `api` corresponde al servicio definido en `compose.yaml`. Docker Compose proporciona resolución de nombres entre los servicios que comparten su red interna. Los puertos se publican en `127.0.0.1` para restringir el acceso al equipo anfitrión durante las pruebas locales.

2.5. Independencia entre componentes

El frontend obtiene la dirección del backend mediante la variable de entorno: `FINPREDICT_API_URL` Durante la ejecución local de Python, la aplicación utiliza por defecto: `http://127.0.0.1:8000/predict` Durante la ejecución con Docker Compose, se configura: `http://api:8000/predict` Esto permite utilizar el mismo código del frontend en ambos entornos sin modificar las instrucciones de conexión.

2.6. Artefacto predictivo

El backend utiliza el archivo:

`model/finpredict_modelo.joblib` El artefacto contiene:

- Pipeline de preprocesamiento y regresión logística.
- Configuración de 15 variables predictoras.
- Umbral operativo de clasificación de 0.75.

Durante la ejecución del contenedor, FastAPI carga el artefacto desde el sistema de archivos interno. No necesita acceder a Google Colab ni a las carpetas del proyecto académico anterior para generar predicciones.

3. Requisitos previos

3.1. Requisitos de software

Para ejecutar FinPredict mediante Docker Compose se requiere:

Componente	Requisito
Sistema operativo	Windows con Docker Desktop o entorno compatible con Docker
Docker	Docker Desktop o Docker Engine funcional
Docker Compose	Versión integrada mediante `docker compose`
Navegador	Chrome, Edge u otro navegador moderno
Terminal	PowerShell o terminal compatible
Código fuente	Carpeta completa `FinPredict-FaseII`
Internet	Necesario para descargar imágenes y dependencias durante la primera construcción

La ejecución contenerizada no requiere instalar Python, Scikit-learn, FastAPI o Streamlit directamente en el equipo anfitrión. Las dependencias se instalan dentro de las imágenes a partir de `requirements.txt`.

3.2. Requisitos de recursos

El equipo debe contar con recursos suficientes para ejecutar Docker y dos contenedores Python simultáneamente. Los requerimientos exactos de CPU, memoria y almacenamiento no han sido determinados mediante pruebas de capacidad. Se recomienda verificar el espacio disponible antes de construir las imágenes, especialmente porque las imágenes incluyen las bibliotecas de machine learning.

3.3. Puertos requeridos

La configuración actual publica los siguientes puertos:

Puerto	Servicio
8502	Streamlit
8002	FastAPI

Ambos puertos deben encontrarse disponibles en el equipo anfitrión. Si alguno está ocupado, será necesario modificar el puerto publicado correspondiente en `compose.yaml`.

3.4. Verificación de Docker

Abrir Docker Desktop y confirmar que el motor esté en funcionamiento. Desde PowerShell, ejecutar:

```
docker --version

docker compose version

docker info
```

Los comandos deben devolver información del entorno Docker sin errores de conexión con el motor.

4. Preparación del proyecto

4.1. Obtener los archivos

Descargar el repositorio de FinPredict – Fase II desde GitHub utilizando la opción de descarga ZIP o mediante Git, una vez que el repositorio esté publicado. Extraer el contenido en una carpeta local con permisos de lectura y escritura. No se requiere una ubicación específica en el disco.

4.2. Verificar la estructura mínima

Antes de ejecutar Docker Compose, comprobar que existan los siguientes archivos y directorios:

```
FinPredict-FaseII/

|

|-- api/

|   |-- main.py

|   `-- schemas.py

|

|-- frontend/

|   `-- app.py

|

|-- model/

|   `-- finpredict_modelo.joblib

|

|-- scripts/

|   `-- validar_entradas.py

|

|-- Dockerfile.api

|-- Dockerfile.frontend
```

```
|-- compose.yaml
|-- requirements.txt
`-- .dockerignore
```

La carpeta `data/`, las pruebas y la documentación se incluyen como parte del repositorio académico, pero no son necesarias para ejecutar los dos servicios de inferencia dentro de Docker.

4.3. Abrir el proyecto en una terminal

Abrir PowerShell en la carpeta raíz `FinPredict-FaseII`, donde se encuentra `compose.yaml`. Ejemplo de navegación en Windows:

```
cd .\FinPredict-FaseII
```

Este ejemplo supone que la terminal está ubicada en el directorio que contiene la carpeta del proyecto.

4.4. Validar la configuración

Ejecutar:

```
docker compose config --quiet
```

Si la configuración es válida, el comando termina sin mostrar errores. Esta validación comprueba la estructura de Docker Compose, pero no demuestra que los servicios puedan iniciarse correctamente.

5. Construcción y despliegue local

5.1. Construcción e inicio de servicios

Desde la carpeta raíz de FinPredict – Fase II, ejecutar:

```
docker compose up -d --build
```

Este comando realiza las siguientes operaciones:

1. Lee la configuración de `compose.yaml`.
2. Construye la imagen del backend mediante `Dockerfile.api`.
3. Construye la imagen del frontend mediante `Dockerfile.frontend`.
4. Instala las dependencias definidas en `requirements.txt`.
5. Crea la red interna de Docker Compose.
6. Crea e inicia los contenedores correspondientes.

Docker puede reutilizar capas de caché existentes para evitar instalaciones innecesarias. La opción `-d` permite ejecutar los servicios en segundo plano.

5.2. Verificación del estado

Ejecutar:

```
docker compose ps
```

Se esperan dos servicios activos:

Servicio	Imagen	Puerto publicado
api	finpredict-api:v1	127.0.0.1:8002
frontend	finpredict-frontend:v1	127.0.0.1:8502

La columna STATUS debe indicar que los contenedores están en ejecución. El estado Up confirma que el proceso permanece activo, pero no garantiza por sí solo que todos los endpoints funcionen correctamente.

5.3. Acceso a la aplicación

Abrir un navegador e ingresar:

Frontend Streamlit: http://127.0.0.1:8502 La aplicación debe mostrar el formulario de evaluación de riesgo hipotecario con 15 variables predictoras. **Documentación FastAPI:** http://127.0.0.1:8002/docs Swagger UI debe mostrar los endpoints:

- GET /health
- POST /predict

5.4. Comunicación entre servicios

El frontend utiliza la variable de entorno FINPREDICT_API_URL, configurada en compose.yaml:

```
environment:

  FINPREDICT_API_URL: "http://api:8000/predict"
```

La dirección utiliza el nombre del servicio api para comunicarse dentro de la red Docker. No es necesario utilizar el puerto publicado 8002 para la comunicación interna entre contenedores.

5.5. Resultado esperado del despliegue

El despliegue local se considera funcional cuando se cumplen las siguientes condiciones:

- Ambos contenedores permanecen activos.
- La API responde correctamente a GET /health.
- Streamlit permite enviar el formulario financiero.
- FastAPI recibe las 15 variables predictoras.
- La solicitud válida genera una predicción.
- El resultado se presenta correctamente en la interfaz.

La verificación detallada de estos puntos se describe en la siguiente sección del manual.

5.6. Alcance de la reproducibilidad

La ejecución mediante Docker Compose permite reconstruir los servicios utilizando el código, los Dockerfiles y las dependencias del repositorio. Durante el desarrollo de FinPredict se confirmó la reconstrucción de ambas imágenes y la conservación del resultado predictivo del registro sintético de referencia. La reconstrucción se realizó utilizando caché de Docker disponible en el equipo de desarrollo. No se ha verificado todavía una instalación completamente limpia en otro equipo.

6. Validación funcional del despliegue

6.1. Objetivo

Establecer un procedimiento para comprobar la disponibilidad del backend, la ejecución del modelo, la comunicación con Streamlit y el rechazo de solicitudes inválidas. Las pruebas descritas fueron realizadas sobre FinPredict ejecutándose mediante Docker Compose.

6.2. Verificación de disponibilidad HTTP

Con los contenedores en ejecución, abrir:

http://127.0.0.1:8002/health También puede ejecutarse desde PowerShell:

```
Invoke-RestMethod -Uri "http://127.0.0.1:8002/health" -Method Get
```

Respuesta esperada:

```
{

  "servicio": "FinPredict API",

  "estado": "operativo",

  "version": "1.0.0"

}
```

Criterio de aceptación:

- Código HTTP 200.
- Estado del servicio: `operativo`.
- Respuesta válida del backend.

Esta prueba comprueba disponibilidad del endpoint, pero no sustituye la validación de una inferencia.

6.3. Verificación de inferencia mediante Swagger

Abrir:

http://127.0.0.1:8002/docs Procedimiento:

1. Desplegar el endpoint `POST /predict`.
2. Seleccionar `Try it out`.
3. Sustituir el JSON de ejemplo por un registro válido.
4. Presionar `Execute`.
5. Revisar el código HTTP y la respuesta JSON.

Registro sintético de referencia:


```
{
  "monto_original": 4784933.89,
  "plazo_meses": 360,
  "tasa_interes_anual": 10.34,
  "pago_mensual": 43198.32,
  "ingreso_mensual": 143552.49,
  "antiguedad_meses": 51,
  "saldo_vencido": 10551.63,
  "dias_atraso": 18,
  "relacion_pago_ingreso": 0.3009235158512402,
  "saldo_vencido_mensualidades": 0.24426019345196756,
  "importe_pagado_3m": 122220.61,
  "numero_pagos_3m": 4,
  "importe_exigible_3m": 129594.96,
  "obligaciones_3m": 3,
  "porcentaje_pago_3m": 0.9430969383377256
}
```


Resultado de referencia:

Indicador	Resultado
HTTP	200
Probabilidad	0.054075
Probabilidad porcentual	5.4075%
Clasificación	0
Nivel de riesgo	BAJO
Umbral operativo	0.75

La probabilidad se presenta redondeada a seis decimales en la respuesta de la API. **Criterio de aceptación:** La API debe aceptar el registro, generar una respuesta estructurada y reproducir los valores de referencia indicados.

6.4. Validación end-to-end desde Streamlit

Abrir:

http://127.0.0.1:8502 Procedimiento:

1. Confirmar que se visualiza la interfaz FinPredict.
2. Conservar los 15 valores predeterminados del formulario.
3. Presionar `Evaluar riesgo hipotecario`.
4. Revisar los resultados presentados.

Resultado esperado:

Indicador	Resultado
Probabilidad estimada	5.4075%
Clasificación	Riesgo bajo
Umbral operativo	75%

Esta prueba valida el flujo:

Usuario → Streamlit → FastAPI → Modelo → Streamlit. **Criterio de aceptación:** La interfaz debe mostrar la predicción esperada sin errores de comunicación entre contenedores.

6.5. Validación de solicitudes inválidas

Las siguientes pruebas permiten comprobar que la API rechaza entradas incompatibles con el contrato de datos. Todas se ejecutan desde Swagger utilizando `POST /predict`.

Prueba	Modificación del registro	Resultado esperado
Restricción financiera	`ingreso_mensual = -1000.0`	HTTP 422
Campo obligatorio	Eliminar `dias_atraso`	HTTP 422
Tipo incorrecto	`dias_atraso = "dieciocho"`	HTTP 422

Mensajes observados:

- Ingreso negativo: `Valor no positivo: ingreso_mensual`.
- Campo faltante: `Field required`.
- Tipo incorrecto: `Input should be a valid integer`.

Criterio de aceptación: Las tres solicitudes deben rechazarse y la API debe informar el motivo correspondiente. El rechazo de entradas inválidas es un resultado correcto de las validaciones, no una falla del servicio.

6.6. Consistencia después de reconstruir imágenes

Desde la raíz del proyecto, ejecutar:

```
docker compose up -d --build
```

Una vez iniciados ambos servicios, repetir la evaluación del registro sintético de referencia desde Streamlit.
Resultado verificado durante el desarrollo:

- Probabilidad: 5.4075%.
- Clasificación: riesgo bajo.
- Umbral operativo: 75%.

La reconstrucción aprovechó las capas de caché de Docker disponibles. Por ello, esta prueba demuestra consistencia en el entorno evaluado, pero no acredita una instalación completamente limpia en otro equipo.

6.7. Evidencia de las pruebas

Los resultados realizados durante el desarrollo están documentados en: `docs/validacion_pruebas.md` Las capturas se encuentran en: `docs/evidencias/` Las evidencias relacionadas con Docker incluyen las pruebas de construcción, disponibilidad, inferencia, integración, validaciones HTTP y arquitectura de contenedores. La existencia de estas pruebas no sustituye evaluaciones adicionales de seguridad, carga, concurrencia o desempeño en producción.

7. Operación y mantenimiento

7.1. Consultar el estado de los servicios

Desde la carpeta raíz de FinPredict – Fase II, ejecutar:

```
docker compose ps
```

El comando muestra el estado de los contenedores, sus imágenes y los puertos publicados. **Resultado esperado:**

- Servicio `api`: estado `Up`.
- Servicio `frontend`: estado `Up`.
- Puerto del backend: 8002.
- Puerto del frontend: 8502.

El estado `Up` confirma que el proceso permanece activo, pero no sustituye las comprobaciones funcionales HTTP.

7.2. Consultar registros de ejecución

Para revisar los registros recientes:

```
docker compose logs --tail=30
```

Para consultar únicamente el backend:

```
docker compose logs --tail=30 api
```

Para consultar únicamente el frontend:

```
docker compose logs --tail=30 frontend
```

Para observar nuevos eventos en tiempo real:

```
docker compose logs -f
```

Presionar `Ctrl + C` para dejar de visualizar los logs. Esto no detiene los contenedores. Los registros permiten identificar errores de inicio, problemas de importación, excepciones y solicitudes HTTP. Durante la validación local se observaron mensajes de inicio satisfactorio para ambos servicios.

7.3. Reiniciar los servicios

Para reiniciar ambos contenedores:

```
docker compose restart
```

Para reiniciar únicamente FastAPI:

```
docker compose restart api
```

Para reiniciar únicamente Streamlit:

```
docker compose restart frontend
```

Después de un reinicio se recomienda ejecutar:

```
docker compose ps
```

Y verificar nuevamente:

http://127.0.0.1:8002/health **Importante:** `docker compose restart` no reconstruye las imágenes ni aplica automáticamente cambios realizados en el código fuente o en los Dockerfiles.

7.4. Detener y eliminar contenedores

Para detener y eliminar los contenedores administrados por Docker Compose:

```
docker compose down
```

Este comando también elimina la red del proyecto creada por Compose. No elimina las imágenes Docker ni los archivos del repositorio. Para iniciar nuevamente FinPredict:

```
docker compose up -d
```

7.5. Reconstruir después de modificar el código

Cuando se modifiquen archivos del backend, frontend, dependencias o Dockerfiles, ejecutar:

```
docker compose up -d --build
```

Docker reconstruirá las imágenes según corresponda y actualizará los servicios cuando sea necesario. Después de reconstruir, se recomienda:

1. Confirmar el estado con `docker compose ps`.
2. Consultar los registros de ejecución.
3. Verificar `GET /health`.
4. Ejecutar una predicción desde Streamlit.
5. Comparar el resultado del caso de referencia.

La reconstrucción no sustituye una estrategia formal de gestión de versiones o reversión de cambios.

7.6. Consultar las imágenes disponibles

Ejecutar:

```
docker images
```

Las imágenes del proyecto utilizan los nombres:

- `finpredict-api:v1`

- `finpredict-frontend:v1`

La etiqueta `v1` identifica la versión definida en la configuración actual. No constituye por sí sola un mecanismo de trazabilidad completa de versiones de código y modelo.

7.7. Consideraciones operativas

La ejecución actual utiliza Docker Compose en un entorno local de desarrollo. No se ha configurado:

- Monitoreo centralizado de infraestructura.
- Alertamiento automatizado.
- Balanceo de carga.
- Alta disponibilidad entre servidores.
- Administración de secretos productivos.
- Estrategias automatizadas de rollback.
- Actualización continua mediante CI/CD.

Estas capacidades deberán evaluarse antes de considerar una operación empresarial productiva.

8. Solución de problemas

8.1. Objetivo

Establecer procedimientos básicos de diagnóstico ante errores de construcción, inicio, comunicación e inferencia en FinPredict. Las acciones descritas corresponden al despliegue local mediante Docker Compose.

8.2. Matriz de diagnóstico

Problema	Posible causa	Acción recomendada
Docker no responde	Motor Docker detenido	Iniciar Docker Desktop y verificar `docker info`
Error en `docker compose config`	Sintaxis incorrecta en YAML	Revisar indentación, servicios y nombres de archivos
Error durante `docker compose up`	Problema de construcción o dependencias	Consultar el mensaje de error y reconstruir
Puerto 8002 ocupado	Otro proceso utiliza el puerto	Identificar el proceso o modificar el puerto publicado
Puerto 8502 ocupado	Otro servicio utiliza el puerto	Liberar el puerto o modificar `compose.yaml`
Streamlit no abre	Frontend detenido o error de inicio	Consultar estado y logs del frontend
API no responde	Backend detenido o error de inicio	Consultar estado y logs de la API
Streamlit no conecta con FastAPI	URL incorrecta o backend no disponible	Verificar `FINPREDICT_API_URL` y la red interna
Error al cargar el modelo	Artefacto ausente o incompatible	Verificar archivo Joblib y dependencias

Problema	Posible causa	Acción recomendada
HTTP 422	Solicitud incompleta o inválida	Revisar el JSON y las reglas de validación
Predicción inesperada	Diferencias en datos o versión del modelo	Comparar el registro, artefacto y umbral operativo

8.3. Problemas de construcción

Si Docker Compose informa un error durante la construcción, ejecutar:

```
docker compose config --quiet
```

Después intentar:

```
docker compose build
```

Revisar las líneas que indican qué instrucción del Dockerfile falló. Posibles causas:

- Falta de conectividad para descargar dependencias.
- Archivo `requirements.txt` inexistente.
- Versión de biblioteca no disponible para la plataforma.
- Archivo fuente ausente.
- Error en una instrucción `COPY`.

No se recomienda modificar las versiones de bibliotecas sin ejecutar nuevamente las pruebas del modelo, debido a posibles incompatibilidades con el artefacto serializado.

8.4. Conflictos de puertos

FinPredict publica los puertos 8002 y 8502 en el equipo anfitrión. Para comprobar si el puerto 8002 está ocupado en Windows:

```
Get-NetTCPConnection -LocalPort 8002 -ErrorAction SilentlyContinue
```

Para revisar el puerto 8502:

```
Get-NetTCPConnection -LocalPort 8502 -ErrorAction SilentlyContinue
```

Si existe un conflicto, identificar el servicio responsable antes de detenerlo. También es posible modificar los puertos publicados en `compose.yaml`, conservando los puertos internos de los contenedores. Ejemplo:

```
ports:

  - "127.0.0.1:8503:8501"
```

Este ejemplo cambia el puerto de acceso al frontend a 8503. Después de modificar `compose.yaml`, ejecutar:

```
docker compose up -d
```

8.5. Diagnóstico de FastAPI

Consultar el estado del backend:

```
docker compose ps api
```

Consultar sus registros:

```
docker compose logs --tail=50 api
```

Verificar disponibilidad:

```
Invoke-RestMethod -Uri "http://127.0.0.1:8002/health" -Method Get
```

Si la API no inicia, revisar posibles errores de importación, lectura del artefacto Joblib o incompatibilidad de dependencias.

8.6. Diagnóstico de Streamlit

Consultar el estado del frontend:

```
docker compose ps frontend
```

Consultar sus registros:

```
docker compose logs --tail=50 frontend
```

Abrir:

http://127.0.0.1:8502 Si la interfaz abre pero no puede generar una predicción, verificar que FastAPI esté disponible.

8.7. Diagnóstico de comunicación entre contenedores

El frontend debe comunicarse con el backend mediante la variable: `FINPREDICT_API_URL` Su valor dentro de Docker Compose es: `http://api:8000/predict` No debe utilizarse `127.0.0.1:8000` para la comunicación entre contenedores independientes. Para consultar la configuración efectiva:

```
docker compose config
```

Para comprobar que la variable esté disponible dentro del frontend:

```
docker compose exec frontend printenv FINPREDICT_API_URL
```

Resultado esperado: `http://api:8000/predict` Si se modifica la configuración, volver a crear los servicios mediante:

```
docker compose up -d
```

8.8. Errores de validación HTTP

FinPredict utiliza respuestas HTTP para diferenciar solicitudes aceptadas y rechazadas.

Código	Interpretación
200	Solicitud procesada correctamente
422	Error en la estructura, tipo o restricciones de entrada
500	Error interno del servicio, si se presenta

Ejemplos de rechazo esperado:

- `ingreso_mensual = -1000.0`: HTTP 422.
- Ausencia de `dias_atraso`: HTTP 422.

- `dias_atraso = "dieciocho"`: HTTP 422.

Un código 422 no implica necesariamente una falla de infraestructura. Puede representar el funcionamiento correcto de los controles de validación de datos.

8.9. Verificación posterior a una corrección

Después de resolver un incidente, ejecutar:

```
docker compose ps
```

Verificar el endpoint:

http://127.0.0.1:8002/health Finalmente, realizar una evaluación desde: http://127.0.0.1:8502 Para el crédito sintético de referencia, el resultado esperado es:

- Probabilidad: 5.4075%.
- Clasificación: riesgo bajo.
- Umbral operativo: 75%.

Si el resultado coincide, se confirma la recuperación funcional del caso evaluado. Esta verificación no sustituye un conjunto completo de pruebas de regresión.

9. Propuesta de despliegue en la nube

9.1. Objetivo

Describir una arquitectura de referencia para una eventual implementación de FinPredict en infraestructura cloud, conservando la separación entre frontend y backend. El procedimiento presentado es una propuesta técnica. No se realizó un despliegue real en AWS ni se validaron costos, capacidad, disponibilidad o seguridad cloud.

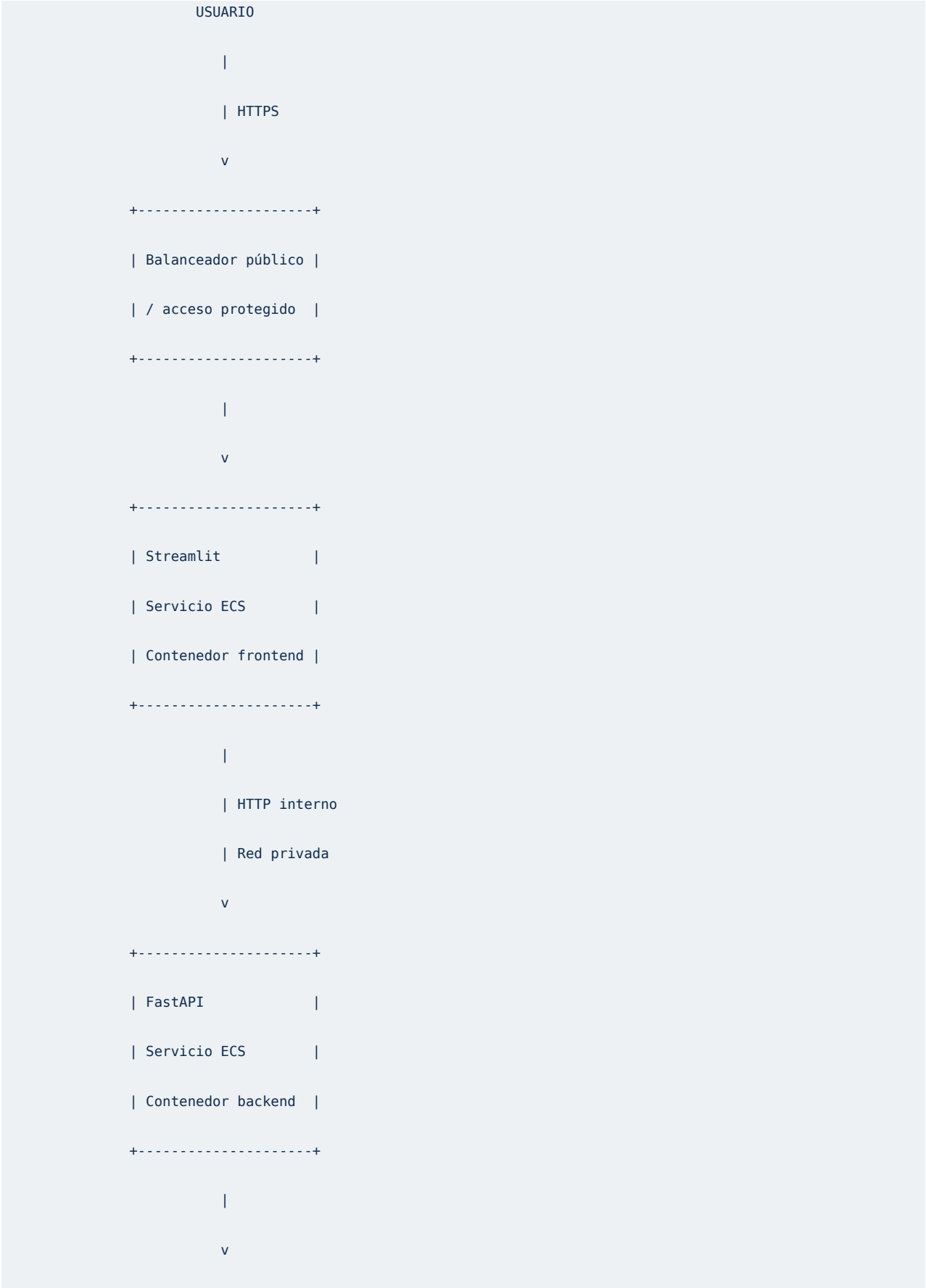
9.2. Plataforma de referencia

Se propone Amazon Web Services (AWS), utilizando servicios administrados para almacenamiento y ejecución de imágenes Docker.

Componente	Servicio AWS propuesto	Función
Registro de imágenes	Amazon ECR	Almacenar las imágenes Docker
Ejecución de contenedores	Amazon ECS con Fargate	Ejecutar frontend y backend
Balanceo de tráfico	Application Load Balancer	Distribuir solicitudes HTTPS
Certificados	AWS Certificate Manager	Administrar certificados TLS
Registros	Amazon CloudWatch Logs	Centralizar logs de ejecución
Secretos	AWS Secrets Manager	Administrar credenciales, si se requieren
Control de acceso	AWS IAM	Gestionar permisos técnicos
Red	Amazon VPC	Aislar y comunicar servicios

La selección es ilustrativa y deberá confirmarse mediante un análisis de requerimientos y costos.

9.3. Arquitectura propuesta



```
            +--------------------+
            | Modelo Joblib      |
            | Pipeline predictivo |
            +--------------------+
        ECR: imágenes de ambos servicios
        CloudWatch: registros operativos
        IAM: permisos de infraestructura
```

El backend no debería exponerse directamente a internet durante una implementación empresarial. La comunicación entre frontend y backend debería realizarse mediante mecanismos internos de descubrimiento de servicios y controles de red.

9.4. Preparación de imágenes

Las imágenes locales de referencia son:

- `finpredict-api:v1`
- `finpredict-frontend:v1`

Para una implementación cloud se propone:

1. Crear repositorios de imágenes en Amazon ECR.
2. Autenticar el cliente Docker ante el registro.
3. Etiquetar las imágenes con identificadores de

versión y, preferentemente, de compilación.

4. Publicar las imágenes en los repositorios ECR.
5. Registrar las referencias de imagen que utilizarán

los servicios de ejecución. Antes de publicar, se deberán revisar dependencias, vulnerabilidades y contenido de las imágenes.

9.5. Configuración del backend

Crear una definición de tarea para FastAPI y configurar un servicio de ejecución en ECS. La tarea deberá:

- Utilizar la imagen del backend almacenada en ECR.
- Ejecutar FastAPI en el puerto interno 8000.
- Incluir el artefacto predictivo dentro de la imagen.
- Registrar logs en CloudWatch.
- Disponer de permisos IAM mínimos necesarios.
- Contar con recursos de CPU y memoria definidos

después de realizar pruebas de capacidad. Se recomienda configurar comprobaciones de salud contra el endpoint: `GET /health` El backend deberá permanecer accesible únicamente para los componentes autorizados.

9.6. Configuración del frontend

Crear una definición de tarea y un servicio independiente para Streamlit. La tarea deberá:

- Utilizar la imagen frontend almacenada en ECR.
- Ejecutar Streamlit en el puerto interno 8501.

- Configurar la dirección del backend mediante

la variable `FINPREDICT_API_URL`.

- Registrar logs en CloudWatch.
- Recibir tráfico únicamente mediante los

componentes de acceso autorizados. La dirección utilizada en Docker Compose: `http://api:8000/predict` no debe trasladarse automáticamente a ECS. En un despliegue cloud se requiere configurar un nombre o dirección interna que realmente resuelva al servicio FastAPI en esa infraestructura.

9.7. Red y acceso

La arquitectura deberá contemplar:

1. Una VPC con segmentación de red adecuada.
2. Acceso público controlado exclusivamente

para los componentes necesarios.

3. Comunicación privada entre Streamlit y FastAPI.
4. Reglas de seguridad que limiten puertos y orígenes.
5. Cifrado TLS para el acceso externo.
6. Autenticación y autorización según los

perfiles de usuario definidos por negocio. La autenticación de usuarios no forma parte del prototipo local actual. Su implementación sería obligatoria antes de utilizar información financiera real.

9.8. Publicación y verificación

Una vez configurada la infraestructura, el procedimiento propuesto de verificación sería:

1. Confirmar que las imágenes estén disponibles

en el registro ECR.

2. Verificar que las tareas ECS se encuentren activas.
3. Revisar el estado de los servicios y sus logs.
4. Comprobar que FastAPI responda a `/health`

desde una ubicación autorizada.

5. Abrir la interfaz Streamlit mediante HTTPS.
6. Ejecutar una inferencia con el registro sintético

de referencia.

7. Comparar la probabilidad, clasificación y umbral

con los resultados de validación local.

8. Enviar solicitudes inválidas para confirmar

los controles HTTP.

9. Documentar resultados y posibles incidencias.

Los criterios funcionales de referencia serían:

Indicador	Resultado esperado
Disponibilidad API	HTTP 200
Probabilidad de referencia	5.4075%

Indicador	Resultado esperado
Clasificación	Riesgo bajo
Umbral operativo	75%
Entradas inválidas	HTTP 422

Estos valores corresponden a pruebas locales previas y no a resultados obtenidos en AWS.

9.9. Requisitos antes de producción

Previo a una implementación empresarial deberán evaluarse y, cuando corresponda, implementarse:

- Validación predictiva con datos reales autorizados.
- Evaluación de sesgo, explicabilidad y estabilidad.
- Seguridad y control de acceso.
- Gestión de secretos y configuración.
- Pruebas de carga, concurrencia y disponibilidad.
- Monitoreo de infraestructura y comportamiento

del modelo.

- Versionado y trazabilidad de modelos e imágenes.
- Procedimientos de actualización y rollback.
- Evaluación de costos y capacidad.
- Cumplimiento normativo y aprobación de negocio.

9.10. Conclusión de la propuesta cloud

La arquitectura contenerizada de FinPredict constituye una base técnica para considerar su despliegue en servicios administrados. Sin embargo, la existencia de imágenes Docker funcionales no demuestra preparación para producción. La propuesta AWS representa una ruta de evolución que requiere diseño detallado, implementación, validación de seguridad y pruebas operativas. **Estado:** Propuesta documental no implementada ni validada en infraestructura cloud.

10. Seguridad, riesgos y limitaciones

10.1. Consideraciones de seguridad

FinPredict fue desarrollado como un prototipo académico y no incorpora todos los controles necesarios para operar con información financiera real. La configuración de Docker Compose publica los servicios en la interfaz local `127.0.0.1`, limitando su exposición de red durante las pruebas. Esta configuración no sustituye mecanismos de autenticación, autorización, cifrado de comunicaciones ni protección de infraestructura.

10.2. Riesgos identificados

Riesgo	Posible impacto	Tratamiento recomendado
Acceso no autorizado	Uso indebido del servicio	Implementar autenticación y autorización

Riesgo	Posible impacto	Tratamiento recomendado
Entradas inválidas	Errores de inferencia	Mantener y ampliar validaciones
Dependencias vulnerables	Exposición técnica	Analizar imágenes y actualizar dependencias de forma controlada
Indisponibilidad del backend	Interrupción de predicciones	Incorporar monitoreo y mecanismos de recuperación
Cambios del modelo	Resultados inconsistentes	Versionar artefactos y ejecutar pruebas de regresión
Deriva de datos	Reducción del desempeño predictivo	Implementar monitoreo estadístico
Uso inadecuado de resultados	Decisiones crediticias incorrectas	Mantener supervisión humana y políticas de uso

10.3. Limitaciones del prototipo

Las pruebas realizadas no incluyen:

- Validación predictiva con datos financieros reales.
- Auditoría de sesgos o equidad algorítmica.
- Pruebas de carga y concurrencia.
- Evaluación formal de capacidad y costos.
- Alta disponibilidad entre servidores.
- Escalamiento horizontal.
- Autenticación de usuarios.
- Despliegue público en infraestructura cloud.
- Monitoreo continuo del desempeño predictivo.
- Reentrenamiento automatizado del modelo.

Por lo tanto, los resultados obtenidos respaldan la viabilidad técnica del prototipo en el entorno evaluado, pero no su utilización productiva.

10.4. Gobierno del modelo

El modelo de FinPredict utiliza un umbral operativo de clasificación de 0.75. Este umbral forma parte de la configuración del prototipo y no constituye una política crediticia autorizada para una institución financiera. Antes de implementar la solución con datos reales, sería necesario definir responsables de aprobación, criterios de monitoreo, políticas de actualización, controles de acceso y procedimientos de suspensión.

10.5. Uso responsable

FinPredict debe interpretarse como una herramienta de apoyo analítico para la priorización preventiva. No deberá utilizarse de manera autónoma para aprobar, rechazar o modificar condiciones de financiamiento. Cualquier aplicación empresarial requeriría validaciones adicionales, supervisión humana y aprobación de las áreas responsables.

11. Evidencias, documentación y referencias

11.1. Documentos del proyecto

El proyecto incluye los siguientes documentos:

Documento	Ubicación	Propósito
README general	`README.md`	Presentación e instrucciones rápidas
Informe de validación	`docs/validacion_pruebas.md`	Pruebas y resultados técnicos
Manual de despliegue	`docs/manual_despliegue.md`	Instalación, operación y diagnóstico
Evidencias	`docs/evidencias/`	Capturas de las pruebas realizadas

11.2. Evidencias de Docker

Las principales evidencias de construcción, despliegue y validación corresponden a:

Evidencia	Descripción
14	Construcción de imagen FastAPI
15	Disponibilidad HTTP de FastAPI en Docker
16	Inferencia del modelo dentro del contenedor
17	Construcción de imagen Streamlit
18	Interfaz Streamlit mediante Docker Compose
19	Predicción end-to-end contenerizada
20	Estado activo de los servicios
21	Rechazo de ingreso negativo
22	Rechazo de campo obligatorio faltante
23	Rechazo de tipo de dato incompatible
24	Arquitectura visible en Docker Desktop

El inventario completo de las 24 evidencias se encuentra en `docs/validacion_pruebas.md`.

11.3. Resultados de referencia

Indicador	Resultado validado
Modelo	Regresión logística

Indicador	Resultado validado
Variables predictoras	15
Umbral operativo	75%
Consistencia histórica	33,603 registros
Clasificaciones históricas coincidentes	100%
Casos extremos	5 de 5 evaluados
Validaciones locales de entradas	15 superadas
Pruebas HTTP automatizadas	5 superadas
Probabilidad del caso de referencia	5.4075%
Clasificación del caso de referencia	Riesgo bajo

Estos resultados corresponden al entorno local y contenerizado validado durante el desarrollo.

11.4. Declaración de uso de IA generativa

Durante el desarrollo de FinPredict se utilizó ChatGPT como herramienta de asistencia para la exploración de alternativas técnicas, estructuración y depuración de código, comprensión de Docker, FastAPI y Streamlit, y elaboración de documentación. Las pruebas fueron ejecutadas durante el desarrollo del proyecto y sus resultados fueron revisados para documentar el comportamiento de la solución.

11.5. Referencias técnicas

- Docker Documentation: https://docs.docker.com/
- Docker Compose: https://docs.docker.com/compose/
- FastAPI: https://fastapi.tiangolo.com/
- Streamlit: https://docs.streamlit.io/
- Scikit-learn: https://scikit-learn.org/
- Amazon ECS: https://docs.aws.amazon.com/ecs/
- Amazon ECR: https://docs.aws.amazon.com/ecr/

12. Conclusión general

FinPredict – Fase II demuestra la integración de un modelo de clasificación hipotecaria con una API REST, una interfaz web y una arquitectura contenerizada mediante Docker Compose. Las pruebas realizadas respaldan la disponibilidad local de los servicios, la comunicación entre componentes, la consistencia de inferencia y el rechazo de entradas inválidas. El presente manual establece procedimientos para construir, ejecutar, validar y administrar el prototipo, así como una propuesta de evolución hacia infraestructura cloud. La solución constituye un entregable académico funcional y reproducible en el entorno evaluado. Su eventual utilización empresarial requeriría validaciones adicionales de seguridad, desempeño, gobierno y cumplimiento normativo.

FinPredict | Fase II FinComercial, S.A. — Analítica inteligente para anticipar el riesgo hipotecario.